# Khaled Shaaban

**Backend Developer**

khaledcfeki@gmail.com | (+20) 128 884 4010
github.com/SupeeerMario | supeeermario.github.io | linkedin.com/in/khaled-elfeki

## SUMMARY

Backend Developer experienced in building REST APIs in Python (Django, DRF), Node.js (Express) and Go. Delivered a freelance OCR system to a client and built three projects that each solve a common production failure: double charges, a worker crashing mid-import, and one slow endpoint stalling the rest. I prove each guarantee with a test that fails when it breaks, and I ship my own work from Docker and CI to a Linux server.

## TECHNICAL SKILLS

**Languages:** Python, Go, JavaScript (Node.js), SQL

**Backend:** Django, Django REST Framework (DRF), Express.js, Gin, REST API design, API-key authentication, OpenAPI / Swagger

**Databases and Queues:** PostgreSQL, MongoDB (Mongoose, transactions), Redis, Redis Streams, Celery

**Cloud and DevOps:** Docker, Docker Compose, GitHub Actions (CI/CD), Git, Gunicorn, Linux, Oracle Cloud VM, S3-compatible object storage (MinIO)

**Testing and Quality:** Django test runner, Go testing with the race detector, concurrency and failure testing, Ruff linting

## PROFESSIONAL EXPERIENCE

**Backend Developer (Freelance)** Sep 2025

Self-Employed, Remote | Arabic OCR Document Processing System

*Node.js, Express, MongoDB, Tesseract, Cloudinary*

- Built and delivered a REST API that extracts Arabic and English text from PDFs and images up to 50 MB, rasterizing each PDF page separately for Tesseract OCR.
- Moved OCR off the request path: uploads return a job ID immediately, and clients poll a MongoDB-backed status endpoint (processing, completed, or failed with a reason).
- Wrote Arabic-aware text cleanup and delivered results as Word documents in Cloudinary, through versioned upload, status and download endpoints agreed with the client.

## PROJECTS

**Multi-Tenant Billing API**

Live API docs: multi-tenant-billing-app.duckdns.org/api/docs/ | github.com/SupeeerMario/multi-tenant-billing-system

*Django, Django REST Framework, PostgreSQL, Celery, Redis, Docker, GitHub Actions*

- Prevented double charges with a unique index on (tenant, idempotency key) instead of a Python check. In a test firing 8 simultaneous payments at one invoice, exactly one charge is recorded; without the index, the same test records 8.
- Recorded every money movement in an append-only double-entry ledger, with tests asserting that the ledger sums to zero and receivables equal open invoices before and after payment.
- Isolated tenants with a custom DRF API-key authentication class that scopes every query, and scheduled monthly invoicing with Celery beat, using unique constraints so no run bills twice.
- Built a GitHub Actions pipeline (Ruff, tests against PostgreSQL 17, Docker build) that runs in about 45 seconds, and deployed to an Oracle Cloud VM on Gunicorn behind a reverse proxy.

**Bulk File Import Pipeline**

github.com/SupeeerMario/bulk-data-import-pipeline

*Node.js, Express, MongoDB, Redis Streams, MinIO (S3 API), zod, Docker Compose*

- Streamed CSV and Excel uploads straight to S3-compatible storage and returned HTTP 202 with a job ID. A background worker parsed a 500,000-row, 30 MB file with memory flat at 120 to 131 MB.
- Committed each 1,000-row chunk and its checkpoint in one MongoDB transaction. After I killed the worker mid-import, a second worker resumed and finished with exactly 500,000 rows, with no gaps or duplicates.
- Built at-least-once processing on Redis Streams consumer groups with idempotent upserts, retries for transient failures, a dead-letter stream after 5 attempts, and a streamed CSV report of rejected rows.

**Uptime Monitor**

github.com/SupeeerMario/uptime-monitor

*Go, Gin, PostgreSQL, pgx, golang-migrate, Docker Compose*

- Built a scheduler that asks PostgreSQL every 5 seconds which monitors are due and dispatches them to a goroutine worker pool over a buffered channel, tracking in-flight probes so none is queued twice.
- Wrote a race-detector test proving a hung endpoint cannot block healthy ones, and confirmed it fails when the pool is cut to one worker.
- Stored every probe as a row, writing the check and the monitor's last-checked time in one SQL statement, and implemented graceful shutdown that saves every in-flight result before the database pool closes.

## EDUCATION

**B.Sc. in Software Industry and Multimedia**, Faculty of Science, Alexandria University Sep 2020 – Jul 2024